

Working with Databases

Shiju Varghese

<https://shjuvar.medium.com>

Working with MongoDB

**MongoDB is document based
NOSQL database**

MongoDB database environments

- **MongoDB Atlas:** The fully managed service for MongoDB deployments in the cloud
- **MongoDB Enterprise:** The subscription-based, self-managed version of MongoDB
- **MongoDB Community:** The source-available, free-to-use, and self-managed version of MongoDB

Document Database

A record in MongoDB is a document, which is a data structure composed of field and value pairs. MongoDB documents are similar to JSON objects. The values of fields may include other documents, arrays, and arrays of documents.

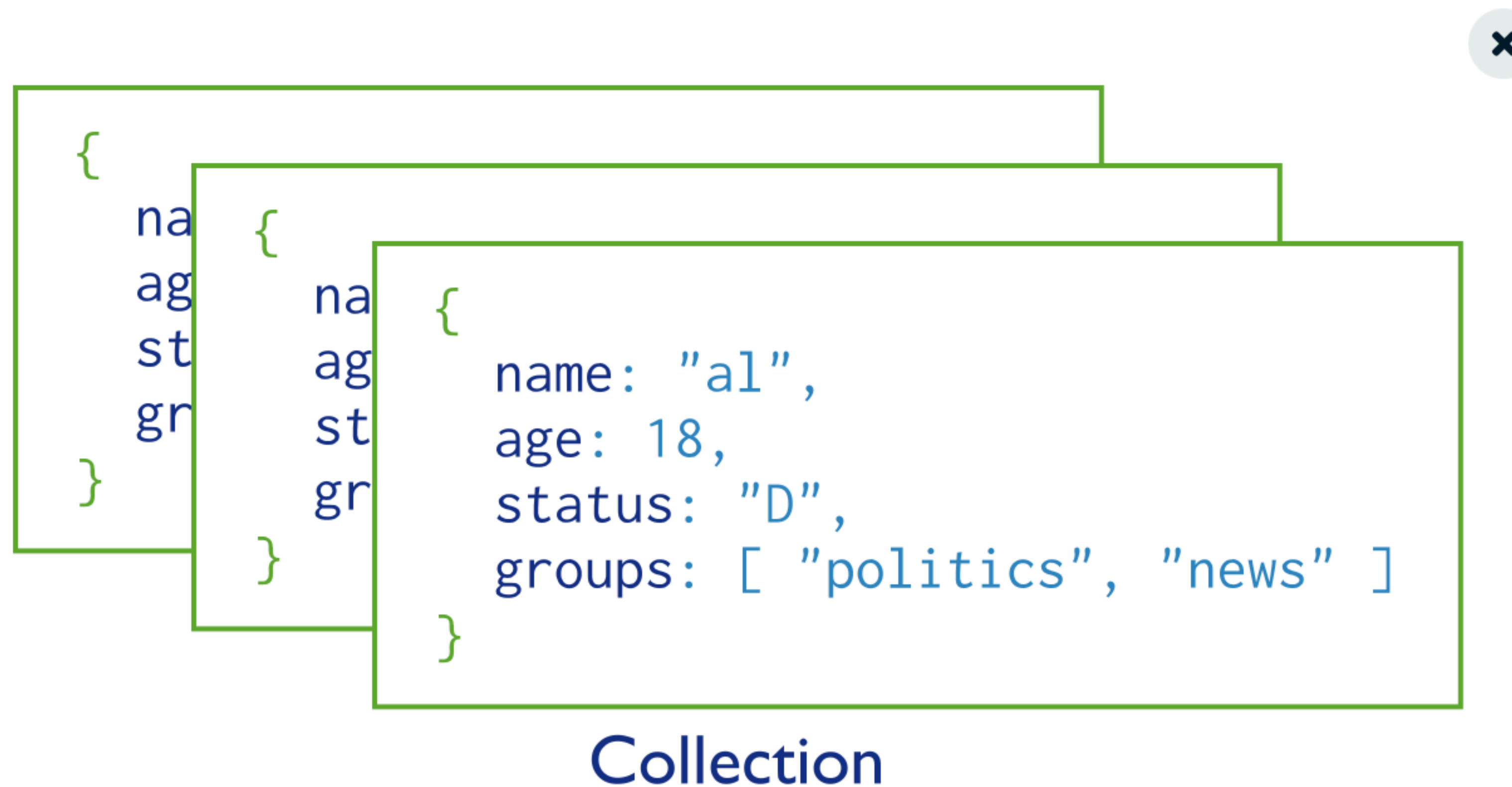
```
{  
  name: "sue",  
  age: 26,  
  status: "A",  
  groups: [ "news", "sports" ]  
}
```



← field: value
← field: value
← field: value
← field: value

Collections

MongoDB stores documents in collections. Collections are analogous to tables in relational databases.



Go Driver for MongoDB

- Source Repository: <https://github.com/mongodb/mongo-go-driver>
- Install Go package: `go get go.mongodb.org/mongo-driver/mongo`

Working with SQL Databases

**Package database/sql provides a generic interface
for SQL (or SQL-like) databases.
The sql package must be used in conjunction with
a database driver: <https://go.dev/wiki/SQLDrivers>**

Approaches

- Write SQL statements in Go code with database/sql package
- Use an ORM
 - GORM: <https://gorm.io/>
 - Ent: <https://entgo.io/> (by Facebook Open Source)
- Compile SQL to type-safe Go code

ent. An entity framework for Go

Install ent code generation tool

```
go install entgo.io/ent/cmd/ent@latest
```

ent

ent is a simple, yet powerful entity framework/ORM for Go, that makes it easy to build and maintain applications with large data-models and sticks with the following principles:

- Easily model database schema as a graph structure.
- Define schema as a programmatic Go code.
- Static typing based on code generation.
- Database queries and graph traversals are easy to write.
- Simple to extend and customize using Go templates.

Supported Databases

- MySQL
- MariaDB
- PostgreSQL
- CockroachDB (preview)
- SQLite
- Gremlin
- TiDB (preview)

Schema

Schema describes the definition of one entity type in the graph, and can contain the following configurations:

- Fields (or properties) in the schema are the attributes of the node
- Edges are the relations (or associations) of entities
- Database specific options, like: indexes or unique indexes

```
func (User) Fields() []ent.Field {
    return []ent.Field{
        field.Int("age"),
        field.String("name"),
        field.String("nickname").
            Unique(),
    }
}

func (User) Edges() []ent.Edge {
    return []ent.Edge{
        edge.To("groups", Group.Type),
        edge.To("friends", User.Type),
    }
}

func (User) Indexes() []ent.Index {
    return []ent.Index{
        index.Fields("age", "name").
            Unique(),
    }
}
```

Type

- All Go numeric types. Like int, uint8, float64, etc.
- bool
- string
- time.Time
- UUID
- []byte (SQL only)
- JSON (SQL only)
- Enum (SQL only)
- Other (SQL only)

```
// Fields of the user.  
func (User) Fields() []ent.Field {  
    return []ent.Field{  
        field.Int("age").  
            Positive(),  
        field.Float("rank").  
            Optional(),  
        field.Bool("active").  
            Default(false),  
        field.String("name").  
            Unique(),  
        field.Time("created_at").  
            Default(time.Now),  
        field.JSON("url", &url.URL{}).  
            Optional(),  
        field.JSON("strings", []string{}).  
            Optional(),  
        field.Enum("state").  
            Values("on", "off").  
            Optional(),  
        field.UUID("uuid", uuid.UUID{}).  
            Default(uuid.New),  
    }  
}
```


ID Field

The id field is builtin in the schema and does not need declaration. In SQL-based databases, its type defaults to int (but can be changed with a codegen option) and auto-incremented in the database.

In order to configure the id field to be unique across all tables, use the `WithGlobalUniqueID` option when running schema migration.

If a different configuration for the id field is needed, or the id value should be provided on entity creation by the application (e.g. UUID), override the builtin id configuration. For example:

```
// Fields of the Group.
func (Group) Fields() []ent.Field {
    return []ent.Field{
        field.Int("id").
            StructTag(`json:"oid,omitempty"`),
    }
}

// Fields of the Blob.
func (Blob) Fields() []ent.Field {
    return []ent.Field{
        field.UUID("id", uuid.UUID{}).
            Default(uuid.New).
            StorageKey("oid"),
    }
}

// Fields of the Pet.
func (Pet) Fields() []ent.Field {
    return []ent.Field{
        field.String("id").
            MaxLen(25).
            NotEmpty().
            Unique().
            Immutable(),
    }
}
```

Validators

A field validator is a function from type `func(T) error` that is defined in the schema using the `Validate` method, and applied on the field value before creating or updating the entity.

The supported types of field validators are string and all numeric types.

```
// Fields of the group.
func (Group) Fields() []ent.Field {
    return []ent.Field{
        field.String("name").
            Match(regexp.MustCompile("[a-zA-Z_]+$")).
            Validate(func(s string) error {
                if strings.ToLower(s) == s {
                    return errors.New("group name must begin with uppercase")
                }
                return nil
            }),
    },
}
```

Built-in Validators

The framework provides a few built-in validators for each type:

- Numeric types:
 - `Positive()`
 - `Negative()`
 - `NonNegative()`
 - `Min(i)` - Validate that the given value is $> i$.
 - `Max(i)` - Validate that the given value is $< i$.
 - `Range(i, j)` - Validate that the given value is within the range $[i, j]$.
- `string`
 - `MinLen(i)`
 - `MaxLen(i)`
 - `Match(regex.Regexp)`
 - `NotEmpty`
- `[]byte`
 - `MaxLen(i)`
 - `MinLen(i)`
 - `NotEmpty`

Optional and Nillable

```
// Fields of the user.
func (User) Fields() []ent.Field {
    return []ent.Field{
        field.String("required_name"),
        field.String("optional_name").
            Optional(),
    }
}
```

```
// Fields of the user.
func (User) Fields() []ent.Field {
    return []ent.Field{
        field.String("required_name"),
        field.String("optional_name").
            Optional(),
        field.String("nillable_name").
            Optional().
            Nillable(),
    }
}
```

Uniqueness

Fields can be defined as unique using the Unique method. Note that unique fields cannot have default values.

```
// Fields of the user.
func (User) Fields() []ent.Field {
    return []ent.Field{
        field.String("name"),
        field.String("nickname").
            Unique(),
    }
}
```

**sqlc: Compile SQL to type-safe code;
catch failures before they happen.**

Database and language support

Language	Plugin	MySQL	PostgreSQL	SQLite
Go	(built-in)	Stable	Stable	Beta
Go	sqlc-gen-go	Stable	Stable	Beta
Kotlin	sqlc-gen-kotlin	Beta	Beta	Not implemented
Python	sqlc-gen-python	Beta	Beta	Not implemented
TypeScript	sqlc-gen-typescript	Beta	Beta	Not implemented

Install sql

```
go install github.com/sqlc-dev/sqlc/cmd/sqlc@latest
```


sqlc Configuration: sqlc.(yaml|yml) or sqlc.json

```
version: "2"
sql:
  - engine: "postgresql"
    queries: "query.sql"
    schema: "schema.sql"
    gen:
      go:
        package: "data"
        out: "data"
        sql_package: "pgx/v5"
```

- **engine:** One of postgresql, mysql or sqlite.
- **schema:** Directory of SQL migrations or path to single SQL file; or a list of paths.
- **queries:** Directory of SQL queries or path to single SQL file; or a list of paths.
- **gen:** A mapping to configure built-in code generators. See gen for the supported keys.

Generating code

sqlc generate